

*****Pornind de la aplicația exemplu din laboratorul 10, să se modifice aceasta astfel încât să se introducă un procesor de flux care să realizeze statistici cu privire la erorile (posibile surse: cele de comunicare, cele de la sistemul de cozi) și tipul acestora care au apărut (dacă au apărut) pentru o licitație până la adjudecare și apoi să scrie aceste informații într-un fișier local. Se vor utiliza microservicii Python și o comunicare prin RabbitMQ. Dacă se consideră necesar se mai pot crea oricâte servicii se dorește conform gândirii studentului. Se vor respecta principiile SOLID pentru microservicii. La prezentare se va arăta diagrama de clase și se va explica aceasta împreună cu gradul de respectare a principiilor. Apoi va urma discuția pe cod.

LAB 10 OKAZII V2 in python GASIT foloeste RABBIT MQ -> LAB5 PT config

Auctioneer.py

```
from RabbitMqConnection import RabbitMqConsumer, RabbitMqProducer
```

```
class Auctioneer:
```

```
    def __init__(self):
        self.producer = RabbitMqProducer(exchange="bidder.direct",
                                         routing_key="messageprocessor.routingkey")
        self.consumer = RabbitMqConsumer(rabbit_queue="bidder.queue")
        self.err = RabbitMqProducer(exchange="bidder.direct",
                                    routing_key="error.routingkey")
```

```
    def receive_bids(self):
```

```
        # se preiau toate ofertele din topicul bids_topic
        print("[Auctioneer] Astept oferte pentru licitatie...")
        try:
```

```
            self.consumer.receive_message()
```

```
        except Exception as e:
```

```
            self.err.send_message("except:{}".format(e))
```

```
        while len(self.consumer.list_msg) != 0:
```

```
            # mesajul e de tip identity:NUME_amount:SUMA
```

```
            msg = self.consumer.list_msg.pop()
```

```
            msg = msg.split("_")
```

```
            identity = msg[0].split(":")[1]
```

```
            amount = int(msg[1].split(":")[1])
```

```
            print("[Auctioneer] {} a licitat {}".format(identity, amount))
```

```
            rabbit_msg = "_".join(msg)
```

```
            # trimit mesaje catre MessageProcessor
```

```

        self.producer.send_message(rabbit_msg)
    try:
        self.consumer.receive_message()
    except Exception as e:
        self.err.send_message("except:{}".format(e))

# bids_consumer genereaza exceptia StopIteration atunci cand se atinge timeout-ul de 10
seconde
# => licitatie se incheie dupa ce timp de 10 secunde nu s-a primit nicio oferta
self.finish_auction()

def finish_auction(self):
    print("[Auctioneer] Licitatie s-a incheiat!")
    # notificam MessageProcessor ca poate incepe procesarea mesajelor
    auction_finished_message = "incheiat"
    self.producer.send_message(auction_finished_message)

def run(self):
    try:
        self.receive_bids()
    except Exception as e:
        self.err.send_message("except:{}".format(e))

if __name__ == '__main__':
    auctioneer = Auctioneer()
    auctioneer.run()
-----
Bidder.py

import os
from random import randint
from uuid import uuid4
from RabbitMqConnection import RabbitMqProducer

class Bidder:
    def __init__(self):
        self.producer = RabbitMqProducer(exchange="bidder.direct",
                                         routing_key="bidder.routingkey")
        self.consumer = open("result.txt", "r") # ofertantul afla rezultatul licitatiei din fisierul
respectiv
        self.my_bid = randint(1000, 10_000) # se genereaza oferta ca numar aleator intre 1000 si
10.000

```

```

self.my_id = uuid4() # se genereaza un identificator unic pentru ofertant

def bid(self):
    # se construiesc mesajul pentru licitare
    print("[ID:{}] Trimit licitatie mea: {}".format(self.my_id, self.my_bid))
    bid_message = "id:{}_amount:{}".format(self.my_id, self.my_bid) # corpul contine id-ul si
suma
    self.producer.send_message(bid_message) # trimitem licitatiea catre Auctioneer

    # exista o sansa din 2 ca oferta sa fie trimisa de 2 ori pentru a simula duplicatele
    if randint(0, 1) == 1:
        self.producer.send_message(bid_message)

def get_winner(self):
    # se asteapta raspunsul licitatiei
    print("[ID:{}] Astept rezultatul licitatiei...".format(self.my_id))

    # asteptam pana cand ne este scris in result.txt castigatorul
    ok = False
    while ok is False:
        if os.stat("result.txt").st_size != 0:
            ok = True
    result = self.consumer.readline()

    # se verifica identitatea castigatorului(ex: winner:ID)
    identity = result.split(":")[1]

    if identity == str(self.my_id):
        print("[ID:{}] Am castigat!!!".format(self.my_id))
    else:
        print("[ID:{}] Am pierdut...".format(self.my_id))

    self.consumer.close()

def run(self):
    self.bid()
    self.get_winner()

if __name__ == '__main__':
    bidder = Bidder()
    bidder.run()

```

BiddingProcessor.py

```
import time
```

```
from RabbitMqConnection import RabbitMqConsumer, RabbitMqProducer
```

```
class BiddingProcessor:
```

```
    def __init__(self):
```

```
        self.producer = open("result.txt", "w")
```

```
        self.consumer = RabbitMqConsumer(rabbit_queue="bidding_processor.queue")
```

```
        self.err = RabbitMqProducer(exchange="bidder.direct",  
                                    routing_key="error.routingkey")
```

```
    def get_processed_bids(self):
```

```
        # se preiau toate ofertele procesate din topicul processed_bids_topic
```

```
        print("[BiddingProcessor] Astept ofertele procesate de MessageProcessor...")
```

```
        # sa aibe timp restu microserviciilor sa-si incheie munca
```

```
        time.sleep(30)
```

```
        # ofertele se stocheaza sub forma de perechi <id:<ID>_amount:<SUMA>>
```

```
        bids = dict()
```

```
        try:
```

```
            self.consumer.receive_message()
```

```
        except Exception as e:
```

```
            self.err.send_message("except:{}".format(e))
```

```
        while len(self.consumer.list_msg) != 0:
```

```
            msg = self.consumer.list_msg.pop()
```

```
            if msg == "incheiat":
```

```
                break
```

```
            msg = msg.split("_")
```

```
            bids[msg[0].split(":")[1]] = msg[1].split(":")[1]
```

```
        try:
```

```
            self.consumer.receive_message()
```

```
        except Exception as e:
```

```
            self.err.send_message("except:{}".format(e))
```

```
        self.decide_auction_winner(bids)
```

```
    def decide_auction_winner(self, bids):
```

```
        print("[BiddingProcessor] Procesez ofertele...")
```

```
        if len(bids) == 0:
```

```
            print("[BiddingProcessor] Nu exista nicio oferta de procesat.")
```

```
            return
```

```

# sortare dupa oferte, descrescator
sorted_bids = sorted(bids.keys(), reverse=False)

# castigatorul este ofertantul care a oferit pretul cel mai mare
winner = bids[sorted_bids[0]]

print("[BiddingProcessor] Castigatorul este:")
print("\t{} - pret licitat: {}".format(winner, sorted_bids[0]))

# se trimite rezultatul licitatiei pentru ca entitatile Bidder sa il preia din topicul
corespunzator
self.producer.write("winner:{}".format(sorted_bids[0]))
self.producer.close()

def run(self):
    self.get_processed_bids()

if __name__ == '__main__':
    bidding_processor = BiddingProcessor()
    bidding_processor.run()
-----
ErrorProcessor.py

import time

from RabbitMqConnection import RabbitMqConsumer

class ErrorProcessor:
    def __init__(self):
        self.consumer = RabbitMqConsumer(rabbit_queue="errors.queue")
        self.file = None # fisierul in care scriem erori

    def errors(self):
        print("[ErrorProcessor] Initializare microserviciu de statistica.")
        time.sleep(5)
        self.file = open("errors.txt", "w")
        try:
            self.consumer.receive_message()
        except Exception as e:
            self.file.write(str(e) + "\n")
        while len(self.consumer.list_msg) != 0:
            msg = self.consumer.list_msg.pop()

```

```

        self.file.write(msg + "\n")
    try:
        self.consumer.receive_message()
    except Exception as e:
        self.file.write(str(e) + "\n")
    self.file.close()

```

```

def run(self):
    self.errors()
    print("[ErrorProcessor] Incheierea activitatii.")

```

```

if __name__ == "__main__":
    processor = ErrorProcessor()
    processor.run()

```

MessageProcessor.py

```
import time
```

```
from RabbitMqConnection import RabbitMqProducer, RabbitMqConsumer
```

```
class MessageProcessor:
```

```

    def __init__(self):
        self.producer = RabbitMqProducer(exchange="bidder.direct",
                                         routing_key="biddingprocessor.routingkey")
        self.consumer = RabbitMqConsumer(rabbit_queue="message_processor.queue")
        self.err = RabbitMqProducer(exchange="bidder.direct",
                                    routing_key="error.routingkey")

```

```

        # ofertele se pun in dictionar, sub forma de perechi <IDENTITATE_OFERTANT,
        MESAJ_OFERTA>

```

```
        self.bids = dict()
```

```

    def get_and_process_messages(self):
        # se asteapta notificarea de la Auctioneer pentru incheierea licitatiei
        print("[MessageProcessor] Astept notificare de la toate entitatile Auctioneer pentru
        incheierea licitatiei...")
        time.sleep(2)

```

```

        # a ajuns prima notificare, se asteapta si celelalte notificari timp de maxim 15 secunde
        try:
            self.consumer.receive_message()

```

```

except Exception as e:
    self.err.send_message("except:{}".format(e))
while len(self.consumer.list_msg) != 0:
    msg = str(self.consumer.list_msg.pop())
    try:
        self.consumer.receive_message()
    except Exception as e:
        self.err.send_message("except:{}".format(e))
    if msg == "incheiat":
        print("[MessageProcessor] Licitatie incheiata. Procesez mesajele cu oferte...")
        break
    split_message = msg.split("_")
    self.bids[split_message[0].split(":")[1]] = split_message[1].split(":")[1]

# sorter dupa valoare
sorted_bids = {k: v for k, v in sorted(self.bids.items(), key=lambda item: item[1])}
self.finish_processing(sorted_bids)

def finish_processing(self, sorted_bids):
    print("[MessageProcessor] Procesarea s-a incheiat! Trimit urmatoarele oferte:")
    for IdBid in sorted_bids:
        identity = IdBid
        amount = sorted_bids[identity]
        print("[MessageProcessor] {} a licitat {}".format(identity, amount))
        rabbit_msg = "id:{}_amount:{}".format(identity, amount)
        self.producer.send_message(rabbit_msg)
    # dupa ce terminam de procesat, anuntam ca MessageProcessor a incheiat munca
    self.producer.send_message("incheiat")

def run(self):
    self.get_and_process_messages()

if __name__ == '__main__':
    message_processor = MessageProcessor()
    message_processor.run()
-----
RabbitMQconnection.py

import pika
from retry import retry

class RabbitMqInterface:

```

```

def __init__(self):
    self.config = {
        'host': '0.0.0.0',
        'port': 5678,
        'username': 'student',
        'password': 'student'
    }
    self.list_msg = []
    self.credentials = pika.PlainCredentials(self.config['username'], self.config['password'])
    self.parameters = (pika.ConnectionParameters(host=self.config['host'],
        pika.ConnectionParameters(port=self.config['port'],
        pika.ConnectionParameters(credentials=self.credentials))

```

```

class RabbitMqProducer(RabbitMqInterface):
    def __init__(self, exchange: str, routing_key: str):
        super().__init__()
        self.config["exchange"] = exchange
        self.config["routing_key"] = routing_key

    def send_message(self, message):
        # automatically close the connection
        with pika.BlockingConnection(self.parameters) as connection:
            # automatically close the channel
            with connection.channel() as channel:
                channel.basic_publish(exchange=self.config['exchange'],
                    routing_key=self.config['routing_key'],
                    body=message)

```

```

class RabbitMqConsumer(RabbitMqInterface):
    def __init__(self, rabbit_queue: str):
        super().__init__()
        self.config["queue"] = rabbit_queue
        self.connection = pika.BlockingConnection(self.parameters)
        self.channel = self.connection.channel()
        self.channel.queue_purge(self.config["queue"])

```

```

@retry(Exception, delay=1, tries=15)
def receive_message(self):
    try:
        result_msg = self.channel.basic_get(self.config['queue'])
        if result_msg[2]:
            self.list_msg.append(result_msg[2].decode("utf-8"))

```



```
        self.channel.basic_ack(result_msg[0].delivery_tag)
    else:
        raise Exception("nici o valoare noua in coada.")
# Don't recover connections closed by server
except pika.exceptions.ConnectionClosedByBroker:
    print("Connection closed by broker.")
# Don't recover on channel errors
except pika.exceptions.ConnectionClosedByBroker:
    print("AMQP Channel Error")
except ValueError:
    print("Value error!")
# Don't recover from KeyboardInterrupt
except KeyboardInterrupt:
    print("Application closed.")
```